

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++ · Java · Python

Juan

Estudiante — 4.º semestre
Doble titulación: Software & Robótica

Bogotá, Colombia · 2026

Índice

I	Fundamentos	2
1	Entrada / Salida Rápida (Fast I/O)	2
1.1	C++ (g++ 8.1, -std=c++17)	2
1.2	Java	2
1.3	Python (3.7.4)	2
2	Formateo de Salida	2
3	Condicionales y Ciclos	2
II	Estructuras de Datos	2
4	Estructuras de Datos (Data Structures)	2
5	Disjoint Set Union (DSU)	2
III	Ordenamiento y Búsqueda	2
6	Ordenamiento y Búsqueda (Sorting & Searching)	2
7	Búsqueda Binaria (Binary Search)	2
IV	Matemáticas	2
8	Matemáticas (Math)	2
9	Teoría de Números (Number Theory)	2
10	Combinatoria (Combinatorics)	2
11	Máscaras de Bits (Bitmasking)	2
12	Geometría (Geometry)	2
V	Técnicas Algorítmicas	2
13	Programación Dinámica (Dynamic Programming)	2
14	Algoritmos Voraces (Greedy)	2
15	Ad-hoc y Simulación	2
VI	Grafos y Árboles	2
16	Grafos (Graphs)	2
17	Árboles (Trees)	3
VII	Cadenas	3
18	Cadenas (Strings)	3

Parte I Fundamentos

1 Entrada / Salida Rápida (Fast I/O)

De más lento (didáctico) a más rápido (para TLE ajustado). Usa el lector por bytes solo cuando haya del orden de 10^6 – 10^7 lecturas.

1.1 C++ (g++ 8.1, -std=c++17)

Rápido: desactiva la sincronización con C (una sola vez al inicio). Punto dulce para casi todo.

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     ios_base::sync_with_stdio(false);
5     cin.tie(nullptr); // no mezclar con scanf
6     int entero; long largo;
7     cin >> entero >> largo;
8     cout << entero << "\n"; // "\n", nunca endl (endl frena)
9 }
```

Más rápido: lector propio por bytes con `fread`. Úsalo solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM]; int puntero = 0, leidos = 0;
4     int leer() {
5         if (puntero == leidos) {
6             leidos = fread(bufer, 1, TAM, stdin);
7             puntero = 0;
8         }
9         return leidos == 0 ? -1 : bufer[puntero++];
10    }
11    int nextInt() {
12        int resultado = 0, c = leer();
13        while (c <= ' ') c = leer(); // saltar blancos
14        bool negativo = (c == '-');
15        if (negativo) c = leer();
16        while (c >= '0' && c <= '9') {
17            resultado = resultado * 10 + (c - '0');
18            c = leer();
19        }
20        return negativo ? -resultado : resultado;
21    }
22 };
```

1.2 Java

Intermedio: `BufferedReader` + `StringTokenizer` + `PrintWriter`. Estándar en CP.

```
1 BufferedReader lector = new BufferedReader(
2     new InputStreamReader(System.in));
3 PrintWriter escritor = new PrintWriter(
4     new BufferedWriter(new OutputStreamWriter(System.out)));
5
6 StringTokenizer separador =
7     new StringTokenizer(lector.readLine());
8 int primero = Integer.parseInt(separador.nextToken());
9 int segundo = Integer.parseInt(separador.nextToken());
10
11 escritor.println(primero + segundo);
12 escritor.flush(); // ! OBLIGATORIO al final, si no, no imprime
```

Leer un arreglo de una línea:

```
1 int n = Integer.parseInt(lector.readLine().trim());
2 int[] arreglo = new int[n];
3 separador = new StringTokenizer(lector.readLine());
4 for (int i = 0; i < n; i++)
5     arreglo[i] = Integer.parseInt(separador.nextToken());
```

1.3 Python (3.7.4)

Intermedio: `readline` + `split` + `map` (más rápido que `input()`).

```
1 import sys
2 entrada = sys.stdin.readline
3 primero, segundo = map(int, entrada().split())
```

Más rápido: leer TODO el `stdin` de golpe y tokenizar. La salida, en UNA sola escritura.

```
1 import sys
2 def main():
3     datos = sys.stdin.buffer.read().split()
4     n = int(datos[0])
5     arreglo = list(map(int, datos[1:1 + n]))
6     sys.stdout.write(f"{sum(arreglo)}\n")
7 main()
```

2 Formateo de Salida

Contenido en desarrollo.

3 Condicionales y Ciclos

Contenido en desarrollo.

Parte II Estructuras de Datos

4 Estructuras de Datos (Data Structures)

Contenido en desarrollo.

5 Disjoint Set Union (DSU)

Contenido en desarrollo.

Parte III Ordenamiento y Búsqueda

6 Ordenamiento y Búsqueda (Sorting & Searching)

Contenido en desarrollo.

7 Búsqueda Binaria (Binary Search)

Contenido en desarrollo.

Parte IV Matemáticas

8 Matemáticas (Math)

Contenido en desarrollo.

9 Teoría de Números (Number Theory)

Contenido en desarrollo.

10 Combinatoria (Combinatorics)

Contenido en desarrollo.

11 Máscaras de Bits (Bitmasking)

Contenido en desarrollo.

12 Geometría (Geometry)

Contenido en desarrollo.

Parte V Técnicas Algorítmicas

13 Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

14 Algoritmos Voraces (Greedy)

Contenido en desarrollo.

15 Ad-hoc y Simulación

Contenido en desarrollo.

Parte VI Grafos y Árboles

16 Grafos (Graphs)

Contenido en desarrollo.

17 Árboles (Trees)

Contenido en desarrollo.

Parte VII Cadenas

18 Cadenas (Strings)

Contenido en desarrollo.